

Проект: "Разработка спецификации требований к программному обеспечению" (сквозной на весь семестр)

Студенты разбиваются на команды по 1-3 человека и выбирают проект (например, «Система для бронирования коворкинга», «Приложение для организации фуд-траков», «CRM для маленькой студии дизайна»). По итогам каждой итерации они добавляют в свой проект артефакты:

Назначение: получить практический опыт применения методов системного анализа и проектирования для создания комплекта документации на программный продукт.

Формат: команды по 1-3 человека. Сквозной проект на весь семестр. После каждой темы студенты применяют полученные знания к своему проекту.

1. **Итерация 1:** Карта стейкхолдеров и список методов выявления требований.
2. **Итерация 2:** Модель бизнес-процессов в BPMN (ключевой процесс).
3. **Итерация 3:** Диаграмма вариантов использования и детализированные спецификации для 2-3 ключевых сценариев.
4. **Итерация 4:** Проектирование архитектуры системы.
5. **Итерация 5:** Нефункциональные требования.
6. **Финальная итерация:** Подготовка итогового документа «Спецификация требований к ПО» и его защита.

Итерация 1: Старт проекта и выявление требований

Артефакт на сдачу: Документ (PPT/Word/Latex), включающий:

1. Карта стейкхолдеров:

- **Что сделать:** выявить всех заинтересованных лиц (стейкхолдеров) проекта.
- **Как оформить:** таблица или диаграмма с классификацией:
 - **Внутренние/внешние**
 - **Роль:** Заказчик, пользователь, администратор, тех. специалист и т.д.
 - **Влияние и интерес:** Высокий/средний/низкий.
 - **Пример для "Коворкинга":** Владелец (высокое влияние, высокий интерес), Арендатор (низкое влияние, высокий интерес), Уборщица (низкое влияние, низкий интерес), Бухгалтер (среднее влияние, средний интерес).

2. План выявления требований:

- **Что сделать:** для каждой группы стейкхолдеров предложить методы сбора информации.
- **Как оформить:** таблица: **Стейкхолдер -> Метод -> Цель опроса.**
- **Пример:** "Владелец коворкинга -> Глубинное интервью -> Выяснить бизнес-цели и финансовые ожидания. Арендаторы -> Онлайн-анкетирование -> Выявить боли и пожелания по бронированию."

3. Предварительное видение продукта:

- **Что сделать:** сформулировать 2-3 ключевых бизнес-требования в формате SMART.
- **Пример:** "Увеличить заполняемость рабочих мест в коворкинге на 25% в течение года после внедрения системы онлайн-бронирования."

Итерация 2: Модель бизнес-процессов в BPMN (ключевой процесс)

Артефакт на сдачу: BPMN-диаграмма в [Draw.io](https://draw.io) или другой программе.

- **Что сделать:** смоделировать **ОДИН ключевой** сквозной бизнес-процесс.
- **Требования к диаграмме:**
 - Используются основные элементы BPMN 2.0: пулы/дорожки, задачи, шлюзы, события (старт, конец, промежуточные).
 - Процесс должен быть смоделирован "как должно быть" (TO-BE), а не "как есть" (AS-IS).
 - Должна быть отражена роль системы и людей.
- **Пример для "Коворкинга":** процесс "Бронирование места клиентом". Должен включать: выбор места/времени на карте, оплату, получение подтверждения, уведомление администратора, доступ в коворкинг по QR-коду.

Итерация 3: Функциональные требования и сценарии

Артефакт на сдачу: Документ с диаграммой и описанием (PPT/Word/Latex).

1. Диаграмма вариантов использования (Use Case Diagram):

- **Что сделать:** показать всех акторов и основные прецеденты (варианты использования) системы.
- **Требования:** использовать отношения (include, extend) между прецедентами.
- **Пример для "Коворкинга":** Акторы: Клиент, Администратор, Бухгалтер. Прецеденты: забронировать место, отменить бронь, оплатить бронь, сгенерировать отчет о доходе.

2. Детализированные спецификации для 2-3 ключевых сценариев:

- **Что сделать:** выбрать самые важные Use Cases и расписать их по шаблону.
- **Шаблон описания Use Case:**
 - **Название:** (пример: Забронировать место)
 - **Актор:** (пример: Клиент)
 - **Предусловия:** (пример: Пользователь авторизован)
 - **Основной поток/сценарий:** (Пошаговое описание успешного сценария без использования элементов UI)
 - **Альтернативные потоки/сценарии:** (пример: "Место уже занято", "Оплата не прошла")
 - **Ошибки:** (пример: "Сервер не отвечает", "Пользователь не ввёл обязательные данные")
 - **Постусловия:** (пример: Бронь создана, списаны деньги, отправлено письмо)

3. Написать по описанным УС функциональные требования по шаблону: «Система должна ...» (должны быть обеспечены полнота требований и их качество (атомарность, проверяемость)

Итерация 4: Проектирование архитектуры и поведения

Задание: Спроектируйте архитектуру вашей системы, используя модель C4. Визуализируйте, из каких крупных частей (контейнеров) она будет состоять, как они взаимодействуют друг с другом и с внешними системами. Детализируйте ключевые сценарии с помощью диаграмм UML.

Артефакт на сдачу: Две диаграммы UML.

1. Диаграмма Контекста (C1) и Диаграмма Контейнеров (C2).
2. Диаграмма Компонентов (C3) для одного ключевого контейнера (опционально, но желательно для сильных команд).
3. Диаграмма Классов (UML) — предыдущее задание.
4. Диаграмма Последовательностей (UML) — предыдущее задание, которая теперь должна логично вытекать из сценариев, описанных на C2/C3.
- 5.
6. **Диаграмма классов (Class Diagram):**
 - **Что сделать:** спроектировать логическую модель данных системы.
 - **Требования:** минимум 5-7 ключевых сущностей (классов) с атрибутами и методами. Показать связи между ними (ассоциации, агрегации, наследование) с указанием мощности связи (1, *, 0..1).
 - **Пример для "Коворкинга":** Классы: User, CoworkingSpace, Booking, Invoice, Workspace. Связь: User *makes* 1..* Booking.
7. **Диаграмма последовательностей (Sequence Diagram):**
 - **Что сделать:** детально описать взаимодействие объектов в рамках **одного** из ранее описанных сценариев.
 - **Требования:** показать акторов, объекты системы, сообщения между ними (вызовы методов, возвраты), циклы, условия.
 - **Пример:** Взаимодействие в сценарии "Забронировать место": Клиент -> Frontend -> BookingController -> PaymentService -> EmailService.

Часть 1: Диаграмма Контекста (Context Diagram - C1)

- **Что это:** Самый высокоуровневый вид. Система предстает как один "черный ящик".
- **Что показать:**
 - Ваша система (например, "Система бронирования коворкинга") в центре.
 - Все люди и **внешние системы**, которые с ней взаимодействуют (акторы из Use Case Diagram + внешние API, сервисы).

- Линии, показывающие характер взаимодействия (например, "Бронирует место", "Отправляет email-подтверждение").
- **Пример для "Коворкинга":**
 - Акторы: Аренатор, Администратор.
 - Внешние системы: Платежный шлюз (Stripe), Email-сервис (SendGrid), СМС-шлюз.
- **Критерий оценки:** Полнота отображения всех внешних связей системы.

Часть 2: Диаграмма Контейнеров (Container Diagram - C2)

- **Что это:** Раскрывает "черный ящик" из C1. Показывает, из каких крупных процессов/приложений/сервисов состоит система.
- **Что показать:**
 - **Контейнеры:** Веб-приложение (Frontend), Серверное API (Backend), Мобильное приложение, База данных, etc.
 - **Технологии (опционально, но рекомендуется):** Указать технологии для каждого контейнера (например, "React", "Node.js API", "PostgreSQL").
 - Связи между контейнерами (например, "Web App взаимодействует с Backend API по HTTPS/JSON").
- **Пример для "Коворкинга":**
 - Контейнеры: Single-Page Application (Angular), Backend API (Java Spring Boot), Реляционная БД (PostgreSQL), Кэш (Redis).
 - Связь: SPA -> Backend API: "Отправляет запрос на бронь".
- **Критерий оценки:** Логичное разделение системы на компоненты, соответствие связей бизнес-логике.

Часть 3: Диаграмма Компонентов (Component Diagram - C3)

- **Что это:** Детализация одного контейнера (обычно Backend). Показывает, из каких компонентов (логических модулей) он состоит.
- **Что показать (на примере Backend):**
 - Выделить ключевые компоненты, отвечающие за бизнес-логику: BookingController, UserService, PaymentProcessor, EmailNotifier.
 - Показать зависимости между ними.
- **Пример для "Коворкинга" (внутри контейнера Backend API):**
 - Компоненты: BookingController -> зависит от BookingService -> зависит от PaymentGatewayAdapter и EmailService.

- **Критерий оценки:** (Для продвинутых команд) Грамотное выделение компонентов и их ответственностей (принцип единственной ответственности - SRP).

Связь с UML (Части 4 и 5)

- **Диаграмма Классов** теперь должна логично детализировать один или несколько ключевых компонентов из C3 (например, внутреннюю структуру BookingService и его сущностей).
- **Диаграмма Последовательностей** должна визуализировать один из потоков взаимодействия, показанных на C2 или C3 (например, последовательность вызовов: Frontend -> BookingController -> BookingService -> PaymentGateway -> EmailService).

Итерация 5: Нефункциональные требования (NFR)

Артефакт на сдачу: Раздел "Нефункциональные требования" в итоговом документе.

- **Что сделать:** сформулировать NFR для проекта, классифицировав их.
- **Требования:** требования должны быть измеримыми и проверяемыми (по SMART).
- **Примеры для "Коворкинга":**
 - **Производительность:** "95% всех поисковых запросов к карте мест должны выполняться менее чем за 1 секунду."
 - **Безопасность:** "Все пароли должны храниться в хэшированном виде с использованием алгоритма bcrypt."
 - **Надежность:** "Система должна обеспечивать uptime не менее 99.5% в течение месяца."
 - **Удобство использования:** "Новый пользователь должен совершить первую бронь не более чем за 3 минуты."

Финальная итерация: Комплектация и защита

Артефакт на сдачу:

1. Итоговый документ "Спецификация требований к ПО":

- Документ (30-40 страниц), собранный из всех предыдущих итераций, приведенный к единому стилю. Должен содержать:
 - Титульный лист, оглавление.
 - Введение (описание проекта, цели).
 - Бизнес-требования.
 - Модели (BPMN, Use Case, Class, Sequence).
 - Детализированные сценарии.
 - Нефункциональные требования.
 - Глоссарий.

2. Презентация и защита (5-7 минут на команду):

- Краткий рассказ о проекте.
- Обоснование ключевых решений (почему именно такие процессы, такие сущности, такие NFR).
- Ответы на вопросы преподавателя.